5. Hafta

Operatör Yükleme

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- Bir tipin dilin yerleşik operatörleriyle (`+`, `==`, `<<` vb.) neden ve nasıl doğal biçimde kullanılabildiğini açıklayabilecek; üye ile serbest (ve gerektiğinde `friend`) operatör fonksiyonu arasındaki farkı ayırt edebileceksiniz;
- Aritmetik (`+`, `-`, `*`) ve bileşik atama (`+=`) operatörlerini const doğruluğuna ve doğru dönüş biçimine (değer vs referans) dikkat ederek yükleyebileceksiniz;
- C++20 üç yönlü karşılaştırma operatörünü (`<=>`) `= default` ile veya elle tanımlayarak altı karşılaştırma operatörünü tek bir tanımdan türetebileceksiniz;
- `operator<<`/`operator>>` akış operatörlerini serbest/`friend` fonksiyon olarak, `operator[]` indeksleme operatörünü üye fonksiyon olarak (const/non-const çiftiyle) yazabileceksiniz;
- Bu operatörlerin tamamını bir araya getirerek yerleşik tipler gibi doğal davranan uçtan uca bir değer tipi (`Vektor2B`) tasarlayıp uygulayabileceksiniz.

5.1 Neden Operatör Yükleme? Üye ve Serbest Operatör Fonksiyonları

1 • Ön-öğretim 📣 Neden Önemli?

Hafta 1–2'de iki noktayı toplamak için `Nokta c = topla(a, b);` yazmak zorundaydınız — matematikte en doğal yazım olan `c = a + b`'yi derleyici sizin tipiniz için tanımıyordu. Onlarca vektör işleminin bulunduğu bir kodda her toplama bir `topla(...)`, her çıkarma bir `cikar(...)` çağrısına dönüşür; ifade matematiksel gösterimden uzaklaşır ve okunması giderek zorlaşır. Bu ünite tam da o boşluğu — "kendi tipiniz neden `+` ile toplanamıyor?" sorusunu — kapatır.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce şunları hatırlayın: Hafta 3'ten sınıf, üye fonksiyon, örtük `this` ve `private` alan kavramları; Hafta 4'ten `const` üye fonksiyon doğruluğu. Çıkış noktası ise Hafta 1–2'nin `geometri::Nokta` yapısı ve onun üzerinde çalışan serbest `topla(const Nokta&, const Nokta&)` fonksiyonudur (`kod_ornekleri/hafta02/nokta.hpp`). Bu ünite aynı `geometri` ad uzayına kız kardeş bir tip ekleyeceği için, bu kavramların taze olması gerekir.

Geçen hafta kaynaklarınızı önce Rule of 3 ile elle, sonra akıllı işaretçilerle (Rule of 0) güvenle kopyalamayı ve taşımayı öğrendiniz; artık bir nesnenin bellek gibi bir kaynağı doğru sahiplenmesini sağlayabiliyorsunuz. Bu hafta bir adım öteye geçip aynı nesneleri dilin *kendi* yerleşik sembolleriyle — `+`, `==`, `<<`, `[]` gibi — doğal biçimde kullanmayı ele alacaksınız. Çıkış noktası, çok tanıdık bir yerden geliyor: Hafta 1–2'nin `geometri::Nokta` yapısı.

O haftalarda `geometri` ad uzayında yalnızca `double x, y` alanları olan bir `Nokta` yapısı ve onun üzerinde çalışan *serbest* bir `topla` fonksiyonu yazmıştınız:

```
namespace geometri {
struct Nokta { double x; double y; };
Nokta topla(const Nokta& a, const Nokta& b); // serbest fonksiyon
}
```

İki noktayı toplamak isteyince şöyle yazıyordunuz:

```
geometri::Nokta c = geometri::topla(a, b);
```

Bu kod çalışır ve okunur. Ama matematikte iki vektörü topladığınızda `topla(a, b)` değil, sadece `a + b` yazarsınız; kodunuz matematiksel gösterime hâlâ yabancıdır. Aradaki fark küçük görünse de, karmaşık bir formülü koda dökerken `topla(carp(a, 3), b)` ile `a * 3 + b` arasındaki okunabilirlik uçurumu hızla büyür. İşte bu haftanın sorusu tam olarak şudur: kendi tipiniz için `a + b` yazmayı mümkün kılabilir misiniz?

5.1.1 Operatör bir fonksiyondur

C++'ın bu soruya cevabı nettir: `+`, `==`, `<<` gibi semboller aslında özel isimli fonksiyonlardır ve siz bu fonksiyonları kendi tipiniz için yazabilirsiniz. Bir operatörü kendi tipiniz için tanımladığınızda, derleyici o sembolü gördüğü yerde sizin yazdığınız fonksiyonu çağırır.

Tanım: Operatör yükleme

Operatör yükleme (*operator overloading*), dilin yerleşik bir sembolünü (`+`, `==`, `<<` ...) kendi tipiniz için, `operator` anahtar kelimesiyle başlayan özel isimli bir fonksiyon (`operator+`, `operator==` ...) yazarak anlamlandırmaktır. Derleyici `a + b` ifadesini, `a`'nın tipine bakıp uygun `operator+` fonksiyonunun çağrısına çevirir.

operatör yükleme: dilin yerleşik bir sembolünü kendi tipiniz için özel isimli bir fonksiyonla anlamlandırma (*operator overloading*).

Buradaki anahtar sözcük **operatör yükleme**dir: aynı `+` sembolü, işlenenlerin tipine göre farklı fonksiyonlara bağlanır. `3 + 5` yazdığınızda derleyici yerleşik tam sayı toplamasını kullanır; `v1 + v2` yazdığınızda ise sizin `Vektor2B` için yazdığınız `operator+` fonksiyonunu çağırır. Sembol aynıdır; anlam, işlenenlerin tipine göre belirlenir.

Analoji

Operatör yüklemeyi bir uzaktan kumandanın "+" tuşuna benzetebilirsiniz. Aynı "+" tuşu televizyona bağlıyken sesi artırır, klimaya bağlıyken sıcaklığı artırır; tuşun üzerindeki sembol değişmez, ama hangi cihazın (hangi tipin) elinde olduğunuza göre yaptığı iş değişir. C++'ta da `+` sembolü `int` için sayısal toplama, `std::string` için birleştirme, sizin `Vektor2B`'niz için ise vektör toplamı anlamına gelir. Bunu doğrudan koda eşleyin: "cihaz" işlenenin tipidir, "tuş" `+` sembolüdür, tuşa basınca çalışan devre ise o tip için yazılmış `operator+` fonksiyonudur. Kumanda tuşun *hangi* cihaza sinyal göndereceğine nasıl cihazdan karar veriyorsa, derleyici de `+`'nın hangi fonksiyonu çağıracağına işlenenin tipinden karar verir.

Önemli bir sınır hemen baştan konmalı: yeni bir sembol icat edemezsiniz (örneğin `**` diye bir operatör C++'ta yoktur), var olan bir operatörün işlenen sayısını da değiştiremezsiniz. Yalnızca *var olan* sembolleri kendi tipiniz için anlamlandırırsınız. Ayrıca operatör yükleme yalnızca sözdizimsel bir kolaylıktır — perde arkasında hâlâ sıradan bir fonksiyon çağrısı vardır; ama bu kolaylık, kodun okunabilirliğini ciddi biçimde artırır.

5.1.2 İki yol: üye mi, serbest mi?

Bir operatör fonksiyonunu yazmanın iki temel yolu vardır ve bu haftanın tamamı boyunca hangi operatör için hangisini seçeceğinize karar vereceksiniz. Bu yüzden ikisini baştan net ayırt edin.

Birinci yol, operatörü sınıfın *içinde* bir üye fonksiyon olarak yazmaktır. Bu durumda sol işlenen örtük olarak `this`'tir; yani `a + b` ifadesi `a.operator+(b)` çağrısına çevrilir — `a` soldaki nesnedir, `b` ise fonksiyona parametre olarak geçer.

Tanım: Üye operatör fonksiyonu

Üye operatör fonksiyonu (*member operator function*), bir operatörü sınıfın içinde tanımlayan üye fonksiyondur. Sol işlenen sınıfın kendi nesnesidir ve fonksiyona örtük `this` olarak geçer; `a + b`, `a.operator+(b)` biçiminde çağrılır.

üye operatör fonksiyonu: sol işlenen sınıfın kendisi olduğunda, örtük `this` ile çalışan operatör fonksiyonu (*member operator function*).

Bir işlemi üye fonksiyonla yürütmek, onu **üye operatör fonksiyonu** olarak yazmak demektir. Bu yol yalnızca bir koşulda mümkündür: sol işlenen zaten sizin tipinizin bir nesnesi olmalıdır. `v1 + v2`'de sol işlenen `v1` bir `Vektor2B`'dir, dolayısıyla `operator+` pekâlâ üye olabilir.

İkinci yol, operatörü sınıfın *dışında*, ad uzayı kapsamında bir serbest fonksiyon olarak yazmaktır. Bu durumda operatörün her iki işleneni de fonksiyona açık parametre olarak geçer; `a + b` ifadesi `operator+(a, b)` biçiminde çağrılır.

Tanım: Serbest operatör fonksiyonu

Serbest operatör fonksiyonu (*free / non-member operator function*), bir operatörü sınıfın dışında, ad uzayı kapsamında tanımlayan fonksiyondur. Örtük `this` yoktur; her iki işlenen de açık parametre olarak geçer, `a + b` ifadesi `operator+(a, b)` biçiminde çağrılır.

serbest operatör fonksiyonu: sınıfın dışında, ad uzayı kapsamında tanımlanan operatör fonksiyonu (*free / non-member operator function*).

Bir operatörü sınıf dışında yazmak, onu **serbest operatör fonksiyonu** olarak yazmak demektir. Peki hangi yolu ne zaman seçersiniz? Belirleyici tek bir kural

vardır: **sol işlenen sizin tipiniz değilse, üye fonksiyon imkânsızdır.** Çünkü üye fonksiyon çağrısında sol işlenen daima örtük `this`'tir — yani soldaki nesnenin *sizin* sınıfınızdan olması zorunludur. Bu koşul sağlanmıyorsa serbest fonksiyon tek seçenektir.

Bu ayrımın en çarpıcı örneğini bu haftanın ilerleyen ünitelerinde göreceksiniz: `std::cout << v` ifadesinde sol işlenen `std::cout` bir `std::ostream`'dir, sizin `Vektor2B`'niz değildir. Dolayısıyla `operator<<` üye *olamaz*; serbest olmak zorundadır. Şimdilik bu kuralı aklınızda tutun, somut örneğini Ünite 4'te işleyeceksiniz.

Analoji

Üye ile serbest operatör fonksiyonu arasındaki farkı, bir işin kurum içi personelle mi yoksa dışarıdan bir danışmanla mı yürütüldüğüne benzetebilirsiniz. Üye fonksiyon kurumun *kendi* personelidir: içeriden çalışır, örtük `this` ile "kendi" nesnesine doğrudan erişir, çünkü zaten kurumun bir parçasıdır. Serbest fonksiyon ise dışarıdan çağrılan bir danışmandır: kurumun bir çalışanı değildir, işlenenlerin ikisini de dışarıdan (parametre olarak) alır ve hizmetini öyle verir. Koda eşleyin: personel (üye) örtük `this` üzerinden sol işleneni "içeriden" bilir; danışman (serbest) böyle bir ayrıcalığa sahip değildir, her iki işlenen de ona kapıdan, açık parametre olarak girer.

5.1.3 `friend`: dışarıdakine verilen erişim izni

Serbest bir operatör fonksiyonunun bir sorunu vardır: sınıfın dışında durduğu için, sınıfın `private` alanlarına doğrudan erişemez — tıpkı dışarıdan herhangi bir kodun eremeyeceği gibi. Peki serbest bir `operator<<`'in `Vektor2B`'nin gizli `bilesenler_` alanını yazdırması gerekiyorsa ne yaparsınız? İşte burada devreye `friend` girer.

Tanım: `friend` bildirimi

Bir *`friend` bildirimi* (*friend declaration*), bir sınıfın, dışarıdaki belirli bir fonksiyona kendi `private` alanlarına doğrudan erişim izni vermesidir. İzin sınıf gövdesinin içine `friend` anahtar kelimesiyle yazılır; yalnızca ismen belirtilen fonksiyon bu izne sahip olur, dışarıdaki başka hiçbir kod olmaz.

friend bildirimi: bir sınıfın, dışarıdaki belirli bir fonksiyona `private` alanlarına doğrudan erişim izni vermesi (*friend declaration*).

Bir sınıfın dışarıdaki bir fonksiyona bu ayrıcalığı tanıması, **friend bildirimi** ile yapılır. Dikkat edin: `friend` kapsüllemeyi *delmez*, aksine kapsüllemenin bir parçasıdır — çünkü izni *sınıfın kendisi* verir. Dışarıdan bir fonksiyon zorla giremez; ancak sınıf ona ismen izin verdiyse girebilir.

Analoji

`friend`'i güvenilir bir komşuya verilen ev anahtarına benzetebilirsiniz. Normalde eve — yani `private` alanlara — yalnızca ev sahibi, yani sınıfın kendi üye fonksiyonları girebilir. Ama ev sahibi, güvendiği belirli bir komşuya (bir `friend` fonksiyona) özel olarak bir anahtar verebilir; o komşu artık gerektiğinde içeri girip işini görebilir. Kritik nokta şudur: anahtarı *ev sahibi* verir, herkese değil, yalnızca ismen belirttiği kişiye. Koda eşleyin: `friend` bildirimi sınıfın *içine* yazılır (anahtarı ev sahibi verir), yalnızca adı geçen fonksiyonu kapsar (yalnızca o komşu), ve o fonksiyona `private` alanlara — eve — girme izni tanır.

Burada bir inceliği baştan vurgulamak gerekir: her serbest operatör fonksiyonu

`friend` olmak zorunda *değildir*. Bir serbest fonksiyon işini yalnızca sınıfın `public` arayüzünü kullanarak görebiliyorsa, `private`'a hiç dokunmasına gerek kalmaz, dolayısıyla `friend` olması da gerekmez. Ünite 2'de `3 * v` için yazacağınız serbest `operator*` tam da böyledir: yalnızca `public` üye `operator*`'ı çağırır, `private` alana hiç bakmaz, bu yüzden `friend` *değildir*. Buna karşılık Ünite 4'ün serbest `operator<<`'i `bilesenler_`'e doğrudan erişeceği için `friend` olacaktır. Kısacası: serbest olmak ile `friend` olmak ayrı iki karardır; `friend`, yalnızca serbest fonksiyonun `private`'a ihtiyacı olduğunda eklenir.

5.1.4 `Vektor2B` iskeletini kurmak

Artık bu haftanın öğretim aracını oluşturmaya başlayabilirsiniz. `Nokta` ile aynı `geometri` ad uzayında, ona kız kardeş bir tip olarak `Vektor2B`'yi tanımlayacaksınız — iki boyutlu, tam sayı bileşenli bir vektör. (`Nokta` bu hafta hiç değişmez; yalnızca yeni bir dosyada, aynı ad uzayına yeni bir tip eklersiniz.) Sınıfın gizli verisi, iki tam sayıyı tutan bir dizidir:

```
42    std::array<int, 2> bilesenler_;
```

Bileşenleri ayrı `int x_, y_` alanları yerine bir `std::array<int, 2>` olarak tutmanın nedeni ileride belli olacak: Ünite 4'te `[]` ile bu diziye konumsal erişim vereceksiniz, Ünite 3'te ise karşılaştırmayı bu dizi üzerinden bedavaya alacaksınız. Nesneyi kurmak için basit bir kurucu yeterlidir:

```
15    explicit Vektor2B(int x = 0, int y = 0);
```

Bu kurucunun iki varsayılan parametresi (`int x = 0, int y = 0`) sayesinde `Vektor2B()`, `Vektor2B(3)` ve `Vektor2B(2, 3)` çağrılarının üçü de geçerlidir; verilmeyen bileşenler sıfır olur. `explicit` anahtar kelimesi ise tek argümanlı `Vektor2B(3)` çağrısının bir `int`'ten sessiz (örtük) bir dönüşüme yol açmasını engeller — bu alışkanlığı Hafta 3'ten tanıyorsunuz. Kurucunun gövdesi, üye başlatma listesiyle diziyi doldurmaktan ibarettir:

```
10 Vektor2B::Vektor2B(int x, int y) : bilesenler_{x, y} {}
```

5.1.5 İlk operatör: üye `operator+`

Şimdi bu haftanın ilk somut operatörünü yazın. `v1 + v2`'nin çalışması için `Vektor2B`'ye bir üye `operator+` eklersiniz. Sol işlenen (`v1`) zaten bir `Vektor2B` olduğu için üye fonksiyon uygundur; bildirimi sınıfın içine şöyle koyarsınız:

```
1  Vektor2B operator+(const Vektor2B& sag) const;
```

Bu tek satırı parça parça okuyun. Dönüş tipi `Vektor2B`'dir, çünkü toplama sonucu yeni bir vektördür. Fonksiyonun adı, bir operatör olduğu için `operator+`'dır. Tek parametresi `const Vektor2B& sag`'dır — sağdaki işlenen, kopyalanmasın diye referansla, değiştirilmesin diye `const`'la alınır. Son `const` ise fonksiyonun *sol* işleneni (örtük `this`, yani `v1`) değiştirmediğini bildirir; toplama iki işleneni de okur ama hiçbirini bozmaz. Sol işlenenin nereye gittiğine dikkat: parametre listesinde görünmez, çünkü o örtük `this`'tir — `v1 + v2`, aslında `v1.operator+(v2)` çağrısıdır. Gövdesi ise iki vektörün karşılıklı bileşenlerini toplayıp yeni bir `Vektor2B` döndürür:

```
14 Vektor2B Vektor2B::operator+(const Vektor2B& sag) const {
15    return Vektor2B(bilesenler_[0] + sag.bilesenler_[0],
16                    bilesenler_[1] + sag.bilesenler_[1]);
17 }
```

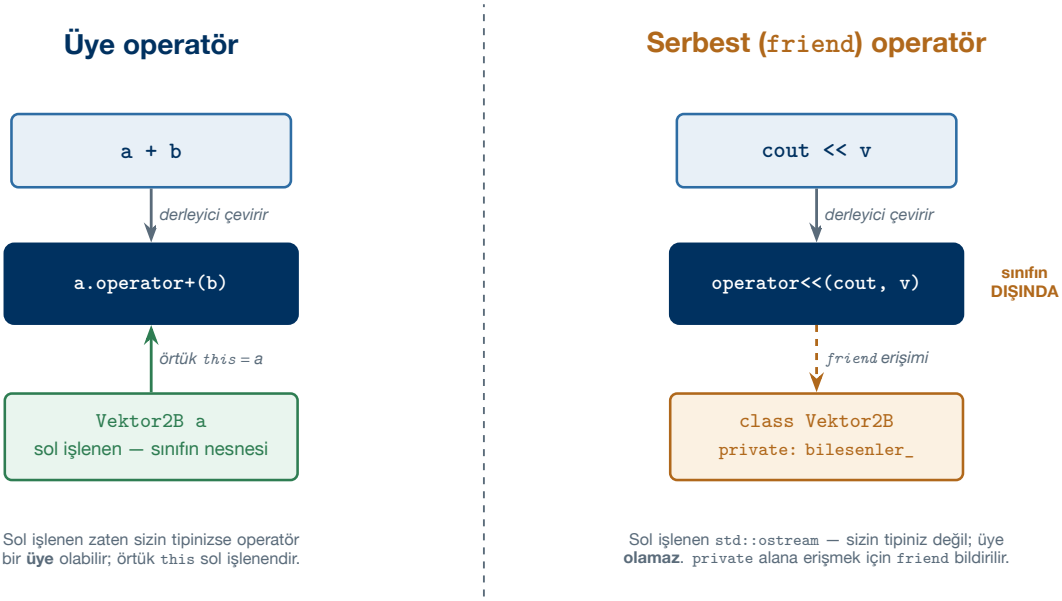
Buradaki `bilesenler_` (nesnenin kendi bileşenleri, yani `this`'e ait olanlar) ile `sag.bilesenler_` (parametrenin bileşenleri) ayrımını görün: fonksiyon üye olduğu için `this`'in bileşenlerine niteleyici olmadan (`bilesenler_`) ulaşır, sağ işlenenin bileşenlerine ise `sag.` önekiyle. İkisi de `private` bir alandır, ama üye fonksiyon kendi sınıfının her nesnesinin `private` alanına erişebildiği için `sag.bilesenler_` de sorunsuz okunur.

Şimdi bu operatörü Hafta 1–2'nin serbest `topla`'sıyla yan yana koyun. İkisi de *aynı* matematiksel işi yapar; fark yalnızca çağrı biçimindedir:

```
geometri::Nokta c = geometri::topla(a, b); // eski: serbest fonksiyon
geometri::Vektor2B c = a + b; // yeni: operator+
```

İkinci satır, `topla`'nın yaptığı işi dilin kendi sözdizimiyle ifade eder; işlem aynı işlemdir, ama artık matematiksel gösterime birebir uyar. Bu haftanın anlatı ekseni tam olarak budur: `topla(a, b)` yazımından `a + b` yazımına geçiş.

Şekil 5.1 bu ünitenin iki temel kararını yan yana özetliyor. Solda, `a + b` ifadesinin üye `operator+` üzerinden `a.operator+(b)`'ye çevrilişi görülüyor: sol işlenen `a` zaten sınıfın nesnesi olduğu için örtük `this` olur. Sağda ise ileride (Ünite 4) somutlaşacak `std::cout << v` durumu var: sol işlenen `std::ostream` olduğundan fonksiyon sınıfın *dışında* serbest olmak zorundadır ve `private bilesenler_`'e erişmek için `friend` bildirilir (şekildeki kesikli ok bu `friend` erişimini gösterir).



Şekil 5.1: Üye operatör (solda): sol işlenen sınıfın nesnesidir, örtük `this` olur. Serbest `friend` operatör (sağda): sol işlenen `std::ostream`'dir, üye olamaz; `private` alana erişmek için `friend` bildirilir.

Bu ünitede gördüğünüz fikir kısaca şudur: operatör yükleme, dilin sembollerini kendi tipiniz için özel isimli fonksiyonlar olarak anlamlandırmaktır; sol işlenen sizin tipinizse operatörü üye, değilse serbest yazarsınız, ve serbest fonksiyon `private`'a erişmesi gerekiyorsa `friend` bildirilir. `Vektor2B`'ye ilk üye `operator+`'ı eklediniz. Gelecek ünitede aritmetiği tamamlayacak (`-`, `*`) ve nesneyi yerinde güncelleyen bileşik atama (`+=`) operatörünü ekleyerek, bu arada `3 * v` örneğiyle serbest — ama `friend` olmayan — bir operatörü de somutlaştıracaksınız.

3 • Katılım — Şimdi Siz Deneyin

`v1` ve `v2` iki `Vektor2B` olsun. (a) `operator+` bir *üye* fonksiyon ise, derleyici `v1 + v2` ifadesini hangi çağrıya çevirir? (b) `std::cout << v1` ifadesini bir *üye* `operator<<` ile yazabilir misiniz — neden?

Geribildirim. (a) `v1 + v2`, `v1.operator+(v2)` çağrısına çevrilir: sol işlenen `v1` örtük `this` olur, `v2` ise tek parametredir. (b) Hayır. `std::cout << v1`'in sol işleneni `std::cout`, yani bir `std::ostream`'dir — sizin `Vektor2B`'niz değildir. Üye çağrıda sol işlenen daima örtük `this` olduğundan, üye `operator<<` sol işleneninin bir `std::ostream` olmasını gerektirirdi; oysa `std::ostream` sizin yazdığınız bir tip değildir, ona üye ekleyemezsiniz. Bu yüzden `operator<<` serbest bir fonksiyon olmak zorundadır.

Kendinizi Sınayın 5.1

`operator+` bir üye fonksiyon olarak yazılabilirken, `operator<<` (`std::cout << v` için) neden üye olamaz, serbest (ve genelde `friend`) olmak zorundadır? Yanıtınızda üye fonksiyon çağrısında sol işlenenin ne olduğunu ve `std::cout << v`'nin sol işleneninin hangi tip olduğunu düşünerek açıklayın.

5 • İzleme — Aktarım — Başka Nerede?

Bu üye/serbest/`friend` ayrımı yalnız `Vektor2B`'ye özgü değildir. Hafta 3'ün `kitaplik::Kitap`'ına bir `operator<<` eklemek isteseydiniz, aynı kuralla o da serbest (ve `friend`) olurdu; çünkü sol işlenen yine `std::ostream`'dir. Herhangi bir değer tipinde — `Kesir`, `Para`, `Tarih` … — "sol işlenen sizin tipiniz mi?" sorusunun cevabı, operatörün üye mi yoksa serbest mi olacağını doğrudan belirler. Bu tek soruyu her operatör için sorma alışkanlığı, bu haftanın geri kalanında ve gelecekteki her yeni tipte işinize yarayacaktır.

5.2 Aritmetik ve Bileşik Atama Operatörleri

1 • Ön-öğretim — Neden Önemli?

Ünite 1'de `v1 + v2`'yi yazabilir hâle geldiniz; ama gerçek kod bununla yetinmez. Vektör farkı (`v1 - v2`), bir tam sayıyla ölçekleme (`v * 3` *ve* `3 * v`) ve bir birikime ekleme (`toplam += v`) de gerekir. `v * 3` çalışırken `3 * v`'nin derlenmemesi can sıkıcı bir asimetridir; ayrıca yeni nesne üreten `+` ile yerinde güncelleyen `+=` farkını yanlış kurmak sarkan referans gibi sinsi hatalar doğurur. Bu ünite aritmetik ailesini eksiksiz ve doğru kurar.

1 • Ön-öğretim — Ön Bilgi

Bu ünite, Ünite 1'in üye/serbest operatör ayrımının ve `Vektor2B` iskeletinin (kurucu, `private std::array<int, 2> bilesenler_`) üstüne oturur. Hafta 4'ün `const` doğruluğu ve değer vs referans dönüş (kopyalama) bilinci de burada doğrudan kullanılacaktır. Ünite 1'de kural olarak tanıtılan "serbest ama `friend`

olmayan fonksiyon" fikri, bu ünitede `3 * v` örneğinde somutlaşacaktır.

Bir önceki ünitede `Vektor2B`'ye ilk üye `operator+`'ı eklediniz ve `a + b` yazımının nasıl mümkün olduğunu gördünüz. Bu ünitede aynı deseni genişleterek aritmetiği tamamlayacaksınız: vektör farkı (`-`), bir tam sayıyla ölçekleme (`*`) ve nesneyi *yerinde* güncelleyen bileşik atama (`+=`). Yol boyunca iki önemli tasarım kararıyla karşılaşacaksınız — bir operatörün ne zaman `const` olması gerektiği, ve sonucu değerle mi yoksa referansla mı döndürmeniz gerektiği. İkisi de Hafta 4'ten tanıdığınız `const` doğruluğu ve kaynak yönetimi alışkanlıklarının doğrudan devamıdır.

5.2.1 Fark ve skalerle çarpma

`operator-`, tıpkı `operator+` gibi bir üye fonksiyondur ve karşılıklı bileşenlerin farkını alıp yeni bir vektör döndürür. Bildirimi `operator+`'ınkiyle neredeyse birebir aynıdır; yalnızca sembol değişir:

```
21 Vektor2B Vektor2B::operator-(const Vektor2B& sag) const {
22     return Vektor2B(bilesenler_[0] - sag.bilesenler_[0],
23                     bilesenler_[1] - sag.bilesenler_[1]);
24 }
```

Skalerle çarpma biraz daha ilginçtir. Bir vektörü bir tam sayıyla ölçeklemek (`v * 3`, yani her bileşeni 3 ile çarpmak) için yine bir üye operatör yazarsınız, ama bu kez parametre bir `Vektor2B` değil, bir `int`'tir:

```
28 Vektor2B Vektor2B::operator*(int katsayi) const {
29     return Vektor2B(bilesenler_[0] * katsayi, bilesenler_[1] * katsayi);
30 }
```

Bu üye `operator*` sayesinde `v * 3` çalışır; derleyici bunu `v.operator*(3)` çağrısına çevirir — sol işlenen `v` bir `Vektor2B` olduğu için üye fonksiyon tam da beklendiği gibi devreye girer. Buraya kadar her şey yolunda görünüyor. Ama tek bir satır bu düzeni bozar.

5.2.2 Simetri sorunu: `v * 3` çalışır ama `3 * v` çalışmaz

Matematikte `3 * v` ile `v * 3` aynı şeydir; doğal olarak ikisini de yazabilmeyi beklersiniz. Ama `3 * v` yazdığınızda kod *derlenmez*. Nedenini adım adım düşünün. `3 * v` ifadesinde sol işlenen `3`, yani bir `int`'tir. Derleyici bu ifadeyi bir üye çağrısı olarak yorumlamaya kalksa, `(3).operator*(v)` araması gerekirdi — yani `int` tipinin, bir `Vektor2B` parametre alan bir üye `operator*`'a sahip olması gerekirdi. Ama `int` yerleşik bir tiptir; ona üye fonksiyon ekleyemezsiniz ve zaten `Vektor2B` alan bir çarpma operatörü yoktur. Sonuç: `v * 3` (sol işlenen `Vektor2B`) çalışır, `3 * v` (sol işlenen `int`) çalışmaz. İşte bu asimetri, Ünite 1'in "sol işlenen sizin tipiniz değilse üye fonksiyon imkânsızdır" kuralının somut sonucudur.

Çözüm de Ünite 1'de tanıttığınız ikinci yoldur: sol işlenen sizin tipiniz olmadığına göre, `3 * v`'yi bir serbest fonksiyon karşılamalıdır. Sınıfın dışına, `int`'i sol, `Vektor2B`'yi sağ işlenen alan bir serbest `operator*` eklersiniz. Bildirimi sınıfın dışında, ad uzayı kapsamında durur:

```
47 Vektor2B operator*(int katsayi, const Vektor2B& v);
```

Bu serbest fonksiyonun gövdesindeki ince zarafeti kaçırmayın: işi sıfırdan yapmaz, zaten var olan üye `operator*`'a devreder:

```
// friend DEGILDIR: yalniz Vektor2B'nin public operator*'ina devreder.
Vektor2B operator*(int katsayi, const Vektor2B& v) {
    return v * katsayi;
}
```

`return v * katsayi;` satırı, sağdaki `v`'nin üye `operator*`'ını çağırır (çünkü `v * katsayi`'da sol işlenen artık bir `Vektor2B`'dir) ve sonucu döndürür. Böylece çarpma mantığını tek bir yerde — üye `operator*`'ta — tutar, serbest fonksiyon yalnızca işlenenlerin sırasını çevirip aynı mantığa yönlendirir. Bunun somut bir yararı vardır: yarın çarpmanın nasıl hesaplandığını değiştirmeniz gerekirse tek bir yeri düzeltirsiniz, `3 * v` de `v * 3` ile otomatik tutarlı kalır.

Şimdi Ünite 1'de bıraktığınız o soruya dönün: bu serbest fonksiyon `friend` olmalı mı? Gövdesine bakın — `bilesenler_` gibi `private` bir alana hiç dokunmuyor, yalnızca `public` olan üye `operator*`'ı çağırıyor. Sınıfın gizli verisine erişmeye ihtiyacı olmadığı için `friend` olmasına *gerek yoktur*. İşte Ünite 1'de söz verilen "serbest ama `friend` değil" örneği tam olarak budur. (Ünite 4'ün `operator<<`'i ise `bilesenler_`'e doğrudan erişeceği için `friend` olacak — iki durumu karşılaştırarak `friend` kararının yalnızca `private`'a erişim ihtiyacına bağlı olduğunu netleştirin.)

5.2.3 Bileşik atama: nesneyi yerinde güncellemek

Şimdiye kadarki üç operatör (`+`, `-`, `*`) hep *yeni* bir vektör üretti; işlenenlerin hiçbirini değiştirmedi. Bazen ise istediğiniz tam tersidir: var olan bir nesnenin üstüne eklemek, onu yerinde güncellemek. `v1 += v2`, "`v1`'e `v2`'yi ekle ve sonucu `v1`'de tut" demektir. Bunu **bileşik atama operatörü** sağlar:

bileşik atama operatörü: bir nesneyi yerinde güncelleyip kendine referans döndüren operatör; `+=`, `-=` gibi (*compound assignment operator*).

```
Vektor2B& Vektor2B::operator+=(const Vektor2B& sag) {
    bilesenler_[0] += sag.bilesenler_[0];
    bilesenler_[1] += sag.bilesenler_[1];
    return *this;
}
```

Bu gövdenin `operator+`'tan üç önemli farkı vardır; üçünü de tek tek görün. **Birincisi**, yeni bir nesne oluşturmaz — doğrudan `this`'in `bilesenler_`'ini günceller (`bilesenler_[0] += sag.bilesenler_[0]`). **İkincisi**, dönüş tipi `Vektor2B` değil, `Vektor2B&`, yani bir *referanstır*; ve döndürdüğü şey `*this`, yani güncellenmiş nesnenin kendisidir. **Üçüncüsü**, fonksiyon `const` *değildir* — çünkü tam da sol işleneni (`this`'i) değiştiriyor.

Neden `*this`'i referansla döndürür? İki gerekçesi var. Birincisi, `a + b` gibi yeni bir nesne üretmediği için ortada değerle döndürülecek yeni bir vektör yoktur; zaten var olan nesnenin kendisini geri verir, kopyalamaya gerek kalmaz. İkincisi, referans döndürmek *zincirlemeyi* mümkün kılar: `(v1 += v2) += v3` gibi bir ifade, ancak `v1 += v2` güncellenmiş `v1`'in kendisini geri verirse çalışır. Bu, yerleşik `int`'lerin `+=`'sinin de davrandığı biçimdir; `Vektor2B`'nizi `int` kadar "doğal" kılmanın bir parçasıdır.

Analoji

`+=` bileşik atamayı bir biriktirme kumbarasına benzetebilirsiniz. Kumbaraya her para attığınızda yeni bir kumbara almazsınız — yani yeni bir nesne oluşturmazsınız; var olan kumbaranın içine doğrudan eklersiniz, kumbara yerinde "büyür". `a + b` ise bunun tersidir: iki kumbaranın içindekini sayıp *üçüncü, yepyeni* bir kumbaraya koymak gibidir, ilk ikisi olduğu gibi kalır. Koda eşleyin: `+=`, `this`'in `bilesenler_`'ini yerinde günceller (aynı kumbara), `operator+` ise iki işlenene dokunmadan yeni bir `Vektor2B` döndürür (yeni kumbara).

5.2.4 `const` doğruluğu: hangi operatör `const` olmalı?

Bu ünitede yazdığınız operatörlerin `const` işaretlerini yan yana koyun; ortaya net bir kural çıkar. `operator+`, `operator-` ve `operator*` salt okunurdur — işlenenleri yalnızca okuyup yeni bir sonuç üretir, hiçbirini değiştirmez — bu yüzden `const` işaretlidir. `operator+=` ise sol işleneni değiştirdiği için `const` *olamaz*. Bu ayrım keyfi değildir; Hafta 4'ten bildiğiniz `const` doğruluğu alışkanlığının doğrudan uygulanmasıdır: bir üye fonksiyon nesneyi değiştirmiyorsa `const` işaretlenir, değiştiriyorsa işaretlenmez.

Bu ayrımın pratik bir sonucu vardır: yalnızca `const` işaretli üye fonksiyonlar bir `const Vektor2B` üzerinde çağrılabilir. Yani elinizde `const Vektor2B sabit(1, 1);` varsa, `sabit + v` (çünkü `operator+ const`'tur) çalışır, ama `sabit += v` (çünkü `operator+= const` değildir) derlenmez — ki bu tam da istediğiniz davranıştır: `const` bir nesneyi değiştirmeye çalışan her satırı derleyici en baştan reddeder.

5.2.5 Neden değerle döndürmek zorunludur

`operator+`, `operator-` ve `operator*`'ın hepsi sonucu *değerle* (by value) döndürür — yani `Vektor2B&` değil, `Vektor2B`. Bunun neden zorunlu olduğunu, yaygın bir tuzağı görerek anlayın.

Ortak Hata / Tuzak

"Kopyalamaktan kaçınmak için `operator+`'ı da `+=` gibi referansla döndürsem daha verimli olmaz mı?" diye düşünmek çok yaygın, ama tehlikeli bir tuzaktır. `+=` kendi nesnesini (`*this`) döndürür — o nesne fonksiyondan sonra da yaşamaya devam eder, bu yüzden ona referans vermek güvenlidir. Oysa `operator+` *yeni* bir sonuç üretir; bu sonuç fonksiyonun içinde oluşturulan yerel bir değişkendir ve fonksiyon bittiği anda kapsam dışına çıkıp yok olur. Eğer bu yerel değişkene referans döndürürseniz, çağıran taraf fonksiyon bittikten sonra artık *var olmayan* bir nesneye bakan bir referans elde eder — buna sarkan (*dangling*) referans denir ve okunması tanımsız davranıştır. Kural nettir: yeni bir nesne üreten operatörler (`+`, `-`, `*`) sonucu *değerle* döndürür; yalnızca var olan nesnesini geri veren operatörler (`+=`) referansla döndürebilir.

5.2.6 Hepsini bir arada görmek

Bu ünitenin operatörlerini gerçek kullanımda izleyin. Aşağıdaki demo `v1=(2,3)` ve `v2=(4,1)` ile başlar, aritmetiği yazdırır, sonra `v1 += v2` ile `v1`'i yerinde günceller:

```
13    Vektor2B v1(2, 3);
14    Vektor2B v2(4, 1);
15    std::cout << "v1=" << v1 << ", v2=" << v2 << '\n';
16    std::cout << "v1+v2=" << (v1 + v2) << '\n';
17    std::cout << "v1-v2=" << (v1 - v2) << '\n';
18    std::cout << "v1*3=" << (v1 * 3) << '\n';
19    std::cout << "3*v1=" << (3 * v1) << '\n';
20    v1 += v2;
21    std::cout << "v1 += v2 sonrasi v1=" << v1 << '\n';
```

Çıktıyı zihninizde yürütün: `v1+v2 (6,4)`, `v1-v2 (-2,2)`, `v1*3 (6,9)` verir. Kritik satır `3*v1`'dir: sonucu yine `(6,9)`'dur — yani serbest `operator*`, üye sürümüyle birebir aynı sonucu üretir; simetri sorunu çözülmüştür. Son olarak `v1 += v2` `v1`'i yerinde `(6,4)`'e günceller (dikkat: bu satırdan sonra `v1` artık `(2,3)` değildir).

Dikkat

Hangi değer tipini yüklüyor olursanız olun, operatörün içinde tutarlı ("kanonik") bir sonuç üretmek sizin sorumluluğunuzdadır. `Vektor2B` için bileşenleri toplamak yeterlidir, çünkü `(6,4)` zaten geçerli ve tek bir gösterimi olan bir vektördür. Ama örneğin bir `Kesir` (pay/payda) tipi yazsaydınız, her aritmetik işlemden sonra sonucu *sadeleştirmeniz* gerekirdi: `2/4 + 0/4` işlemi `2/4` verir, ama bunu `1/2`'ye indirmek (gcd ile ortak böleni ayıklamak) operatörün kendi görevidir — aksi hâlde aynı sayının birden çok gösterimi ortaya çıkar ve karşılaştırma gibi işlemler tutarsızlaşır. Bu `Kesir` fikri bu haftanın kodunda uygulanmaz; yalnızca "operatörün içi tutarlı bir sonuç üretmelidir" ilkesini hatırda tutmanız için bir alıştırma fikri olarak burada durur.

Bu ünitede gördüğünüz fikir kısaca şudur: aritmetik operatörleri (`-`, `*`) üye olarak, yeni nesne ürettikleri için `const` ve değerle döndürerek yazarsınız; sol işlenen sizin tipiniz olmayan `3 * v` için serbest — ama yalnızca `public` arayüzü kullandığı için `friend` olmayan — bir operatör eklersiniz; `+=` ise nesneyi yerinde güncellediği için `const` değildir ve zincirlenmeye izin vermek için `*this`'i referansla döndürür. Gelecek ünitede iki vektörü *karşılaştırmayı* ele alacaksınız; C++20'nin tek bir tanımdan altı karşılaştırma operatörünü türeten `<=>` operatörünü göreceksiniz.

3 • Katılım — Şimdi Siz Deneyin

Üye `operator*(int) const` sayesinde `v * 3` çalışıyor, ama `3 * v` derlenmiyor. (a) Bunu düzeltmek için nasıl bir fonksiyon eklersiniz? (b) Bu fonksiyonun `friend` olması gerekir mi?

Geribildirim. (a) Sınıfın dışında, ad uzayı kapsamında serbest bir `Vektor2B operator*(int katsayi, const Vektor2B& v)` fonksiyonu eklersiniz; `3 * v`'de sol işlenen `int` olduğundan üye olamaz. (b) Hayır. Bu serbest fonksiyon işi `v * katsayi` ile üye `operator*`'a devreder; yalnızca `public` arayüzü kullanır, `private` olan `bilesenler_`'e hiç dokunmaz — dolayısıyla `friend` gerektirmez. Bu, Ünite 1'de söz verilen "serbest ama `friend` değil" durumunun tam da kendisidir.

Kendinizi Sınayın 5.2

`operator+` neden bir referans (`Vektor2B&`) değil, değer (`Vektor2B`) döndürür? Yanıtınızda `operator+`'ın ürettiği sonucun fonksiyon içinde nerede oluştuğunu ve fonksiyon bittikten sonra ona referans döndürmenin neden tehlikeli olduğunu (`operator+=`'nin neden bu sorundan muaf olduğunu da belirterek) açıklayın.

5 • İzleme — Aktarım — Başka Nerede?

Bu ünitenin "yeni nesne → değerle döndür / yerinde güncelle → `*this`'i referansla döndür" kalıbı her değer tipinde geçerlidir. Bir `Kesir(pay, payda)` için de `operator+` yeni, sadeleştirilmiş bir kesir oluşturup değerle döndürür (her işlemden sonra gcd ile sadeleştirmek sizin sorumluluğunuzdadır); `operator+=` ise var olan kesri yerinde günceller. Kitaplık projesinde bir okuyucunun biriken "gecikme günü" sayısı da bir `+=` ile modellenebilir — her yeni gecikme, var olan

toplamın üstüne yerinde eklenir. Kalıbı bir kez kurunca, hangi tip olursa olsun aynı kararı güvenle verirsiniz.

5.3 Karşılaştırma ve C++20 Üç Yönlü Karşılaştırma (<=>)

1 • Ön-öğretim — Neden Önemli?

Bir `std::vector<Vektor2B>`'yi `std::sort` ile sıralamak ya da vektörleri bir `std::set<Vektor2B>` içinde tutmak isterseniz, derleyici `<` operatörünü arar — yoksa kod derlenmez. Altı karşılaştırma operatörünü (`==`, `!=`, `<`, `<=`, `>`, `>=`) elle yazmak hem sıkıcı hem de tutarsızlığa (birinde `<`, diğerinde `<=` mantığını kaçırmaya) açıktır. Bu ünite, altı operatörü tek satırdan üreten C++20 aracını tanıtır.

1 • Ön-öğretim — Ön Bilgi

Bu ünite `Vektor2B`'nin `private std::array<int, 2> bilesenler_` alanını (Ünite 1), `const` üye fonksiyon kavramını (Hafta 4) ve Ünite 2'nin değer semantiğini varsayar. Ayrıca sözlüksel (lexicographic) sıralama sezgisini hatırlayın: tıpkı bir sözlükte kelimelerin önce ilk harfe, o eşitse ikinci harfe göre sıralanması gibi, çok bileşenli bir değer de bileşenlerine soldan sağa bakılarak sıralanır.

Aritmetiği tamamladınız; artık `Vektor2B`'leri toplayıp çıkarabiliyorsunuz. Bu ünitede farklı bir soruyu ele alacaksınız: iki vektörü nasıl *karşılaştırırsınız*? `v1 == v2`, `v1 < v2` gibi ifadelerin çalışması için ne yazmanız gerekir? Cevap yolunda, C++20'nin en zarif yeniliklerinden biriyle — üç yönlü karşılaştırma operatörü `<=>` ile — tanışacaksınız.

5.3.1 Eski dünya: altı operatörü elle yazmak

C++17 ve öncesinde, bir tipin tam anlamıyla karşılaştırılabilir olması için *altı* ayrı operatör yazmanız gerekirdi: `==`, `!=`, `<`, `<=`, `>`, `>=`. Bu altı operatör aslında aynı temel soruyu — "sol işlenen sağdan küçük mü, ona eşit mi, yoksa büyük mü?" — farklı biçimlerde sorar; ama her birini ayrı ayrı yazmak zorundaydınız.

Bunun iki somut sorunu vardı. **Birincisi** tekrardır: altı fonksiyonun mantığı büyük ölçüde aynıdır, yalnızca son karşılaştırma değişir; aynı fikri altı kez yazmak sıkıcı ve hataya açıktır. **İkincisi**, ve daha sinsisi, tutarsızlık riskidir: altısını elle yazarken birinde bir bileşeni unutmak ya da bir işareti ters koymak kolaydır; sonuçta `a < b` ile `a >= b` birbiriyle çelişebilir ve bu tür hatalar fark edilmesi çok güç, sinsi hatalardır. Kısacası, altı operatörü elle tutmak hem yorucu hem de tehlikeliydi.

5.3.2 Yeni dünya: tek tanımdan altı operatör

C++20 bu sorunu tek bir operatörle çözer. Fikir şudur: altı sorunun hepsi tek bir temel bilgiden — "sol küçük mü, eşit mi, büyük mü?" — türetilebilir. O hâlde bu tek bilgiyi üreten *tek* bir operatör yazın, derleyici geri kalan altısını ondan türetsin.

Tanım: Üç yönlü karşılaştırma operatörü (<=>)

C++20'nin *üç yönlü karşılaştırma operatörü* (*three-way comparison / spaceship operator*), `<=>` sembolüyle yazılır ve iki işleneni karşılaştırıp "küçük / eşit / büyük" bilgisini tek bir sonuçta döndürür. Bu tek tanımdan derleyici altı karşılaştırma operatörünü (`<`, `<=`, `>`, `>=` ve — `==` ile birlikte tanımlandığında — `==`, `!=`) otomatik türetir.

spaceship: `<=>` ile "küçük/eşit/büyük" tek sonuçtan altı karşılaştırma operatörünü türeten C++20 operatörü; üç yönlü karşılaştırma (*three-way comparison / spaceship operator*).

Sembolün görünüşü (`<`, `=`, `>` yan yana) bir uzay gemisini andırdığı için ona **spaceship** takma adı yakıştırılmıştır. En güzel yanı ise çoğu durumda gövdesini bile yazmanıza gerek olmamasıdır: derleyiciden varsayılan bir tanım isteyebilirsiniz. `Vektor2B` için karşılaştırma bildirimleri şöyledir:

```
16 bool operator==(const Vektor2B&) const = default;
17 auto operator<=>(const Vektor2B&) const = default;
```

Bu iki satır bu ünitenin kalbidir; ikisini de dikkatle okuyun. İlk satırdaki (`operator<=>` bildirimi) `= default`, "bu operatörün gövdesini sen yaz" demektir — derleyici, üç yönlü karşılaştırmayı sınıfın üyelerini bildirim sırasına göre tek tek karşılaştırarak kendisi üretir. `Vektor2B`'nin tek üyesi `bilesenler_` (bir `std::array<int, 2>`) olduğu için, karşılaştırma dizinin elemanları üzerinden **sözlüksel sıralama** biçiminde işler: önce 0. bileşenlere bakılır; eşitlerse 1. bileşenlere bakılır. Tıpkı bir sözlükte kelimelerin önce ilk harfe, eşitse ikinci harfe göre sıralanması gibi.

sözlüksel sıralama: elemanları soldan sağa sırayla karşılaştıran, ilk farklılığın belirlediği sıralama (*lexicographic ordering*).

İkinci satır, `bool operator==(const Vektor2B&) const = default;`, ilk bakışta gereksiz görünebilir — "`<=>` zaten eşitliği de kapsamıyor mu?" Kapsar, ama ayrıca bir `==` tanımlamanın pratik bir gerekçesi vardır: yalnızca "eşit mi?" sorusunu yanıtlamak, tam bir üç yönlü sonuç (küçük mü / eşit mi / büyük mü) hesaplamaktan genelde daha ucuzdur. İki vektörün eşit olup olmadığını anlamak için bileşenlerin farklı olduğu ilk yerde durup "eşit değil" diyebilirsiniz; oysa sıralama için hangi tarafın büyük olduğunu da belirlemeniz gerekir. Bu yüzden `<=>`'i `= default`'larken `==`'i de ayrıca `= default`'lamak yaygın ve önerilen bir kalıptır — ikisini birlikte yazarsınız, derleyici ikisini de üretir.

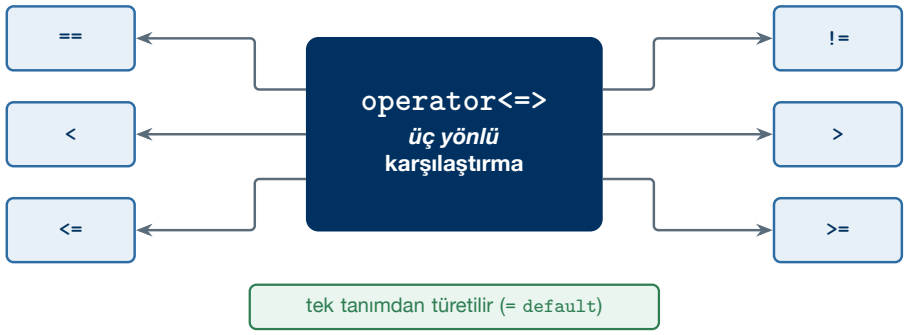
Analoji

Spaceship operatörünü tek bir terazi ölçümüne benzetebilirsiniz. İki nesneyi terazinin iki kefesine koyduğunuzda, terazi size tek seferde "sol daha ağır", "eşit" ya da "sağ daha ağır" der. Bu *tek* ölçümden, aklınıza gelebilecek altı sorunun hepsini cevaplayabilirsiniz: eşit mi? (kefe dengede mi), küçük mü? (sol yukarıda mı), büyük mü? (sol aşağıda mı), küçük-eşit mi, büyük-eşit mi, eşit değil mi. Ayrı ayrı altı farklı ölçüm yapmanıza gerek yoktur; bir ölçüm hepsine yeter. Koda eşleyin: `<=>`, teraziyi bir kez okumaktır (tek karşılaştırma); `<`, `<=`, `>`, `>=` operatörleri ise bu tek okumadan türetilen altı ayrı sorudur — derleyici bunları sizin için otomatik üretir.

Şekil 5.2 bu türetmeyi görsel olarak özetliyor: merkezdeki tek `operator<=>` tanımından altı karşılaştırma operatörü (`==`, `!=`, `<`, `<=`, `>`, `>=`) oklarla ayrışıyor; altta "tek tanımdan türetilir (`= default`)" etiketi bu fikri özetliyor.

5.3.3 Neden karşılaştırma? Sıralama ve kaplar

Burada haklı bir soru akla gelir: iki vektörün "küçüğü" ya da "büyüğü" matematiksel olarak ne anlama gelir? Doğrusu, *hiçbir* geometrik anlamı yoktur — `(1, 5)`'in



Şekil 5.2: Tek bir `operator<=>` tanımından altı karşılaştırma operatörü otomatik türetilir; `= default` bu türetmeyi bileşen sırasına göre bedavaya verir.

`(2, 2)`'den "küçük" olması geometride anlamsızdır. Ama pratikte buna yine de ihtiyacınız olur. Bir `std::vector<Vektor2B>`'yi `std::sort` ile sıralamak ya da vektörleri bir `std::set<Vektor2B>` içinde tutmak isterseniz, bu standart kütüphane araçları elemanları düzene sokabilmek için tutarlı bir sıra ölçütüne ihtiyaç duyar. Aradıkları şey bir **toplam sıralama**dır: kümedeki herhangi iki eleman için "hangisi önce gelir?" sorusunun tutarlı bir cevabı olmalıdır. `<=> = default`, bu toplam sıralamayı bileşen sırasına göre size bedavaya verir — geometrik bir anlam taşımasa da, `std::sort` ve `std::set`'in ihtiyacı olan tutarlılığı sağlar.

toplam sıralama: bir kümedeki her iki eleman için tutarlı bir küçük/eşit/büyük ilişkisi tanımlayan sıralama (*total ordering*).

Karşılaştırmayı ve sıralamayı gerçek kullanımda görün:

```
25  Vektor2B a(2, 3);
26  Vektor2B b(2, 3);
27  Vektor2B c(4, 1);
28  std::cout << std::boolalpha;
29  std::cout << "(2,3) == (2,3) -> " << (a == b) << '\n';
30  std::cout << "(2,3) < (4,1) -> " << (a < c) << '\n';
31
32  std::vector<Vektor2B> vektorler{Vektor2B(3, 1), Vektor2B(1, 5), Vektor2B
        (2, 2)};
33  std::sort(vektorler.begin(), vektorler.end());
34  std::cout << "siralanmis: ";
35  for (const auto& v : vektorler) {
36     std::cout << v << ' ';
37  }
38  std::cout << '\n';
```

Çıktıyı yürütün: `(2,3) == (2,3)` `true` verir (bileşenler birebir aynı). `(2,3) < (4,1)` ise `true` verir — ve buradaki inceliği kaçırmayın: sözlüksel karşılaştırma önce 0. bileşene bakar, `2 < 4` olduğu için 1. bileşene (`3` ile `1`) hiç bakılmadan karar verilir. En öğreticisi `std::sort` çağrısıdır: `{(3,1), (1,5), (2,2)}` listesi, sözlüksel sıraya göre `(1,5), (2,2), (3,1)` biçiminde sıralanır — yani 0. bileşene göre (1, 2, 3) artan sırada. `std::sort`'un tek satırla çalışması, `<=> = default`'ın ürettiği toplam sıralama sayesindedir; siz tek bir karşılaştırma bile yazmadınız.

5.3.4 `= default` ne zaman yanlış sonuç verir?

`= default`'ın gücü, aynı zamanda dikkat etmeniz gereken sınırıdır. Derleyici karşılaştırmayı üyelerin *bildirim sırasına* göre üretir; bu, yalnızca bildirim sırasındaki karşılaştırma *anlamsal* sıraya denk düştüğünde doğru sonuç verir. `Vektor2B` için bu sorun yoktur: bileşenleri sözlüksel karşılaştırmak makul ve tutarlı bir sıradır. Ama her tipte bu şans yaver gitmez.

Dikkat

`= default`, yalnızca üyelerin bildirim sırasındaki karşılaştırması anlamsal/matematiksel sırayla örtüştüğünde doğrudur. Ünite 2'de anılan `Kesir(pay, payda)` tipini düşünün: üye sırasına göre (önce `pay`, sonra `payda`) sözlüksel karşılaştırma, kesirlerin *sayısal* sırasıyla örtüşmez. `1/2` ile `1/3`'ü karşılaştırın: sözlüksel sırada payları eşit (`1 == 1`), sonra paydalar karşılaştırılır ve `2 < 3` olduğu için `1/2 < 1/3` çıkar — oysa sayıca `1/2 > 1/3`'tür. Yani `= default` burada yanlış sonuç verir; böyle bir tipte `<=>`'i *elle*, doğru matematiksel mantıkla (örneğin çapraz çarpma: `a/b` ile `c/d` için `a*d` ile `c*b`'yi karşılaştırarak) yazmanız gerekir. İşte bu, `Kesir`'in bu haftanın birincil örneği olmamasının da gerekçesidir; `Vektor2B` için `= default` doğru, `Kesir` için yanlıştır. `Kesir` bu hafta kod olarak uygulanmaz; not yalnızca bir alıştırma fikri olarak burada durur.

Son olarak kısa bir teknik ayrıntı. `<=>`, karşılaştırmanın ne kadar "güçlü" olduğunu belirten bir sonuç tipi döndürür. `Vektor2B` gibi tam sayı alanlı tiplerde bu tip genelde `std::strong_ordering`'dir — her iki eleman için mutlaka net bir küçük/eşit/büyük ilişkisi vardır. Buna karşılık `double` ya da `float` gibi kayan noktalı alanlar söz konusuysa `std::partial_ordering` üretilir, çünkü kayan noktada `NaN` gibi *hiçbir* şeyle karşılaştırılamayan özel değerler olabilir. `Vektor2B`'nin bileşenleri `int` olduğu için bu haftanın örnekleri hep `strong_ordering` ile çalışır; ayrıntısına girmenize bu hafta gerek yok, yalnızca bu iki adı duymuş olun yeter.

Bu ünitede gördüğünüz fikir kısaca şudur: C++20'nin `<=>` operatörü, tek bir tanımdan altı karşılaştırma operatörünü türetir; `= default` ile yazdığınızda derleyici karşılaştırmayı üye sırasına göre sözlüksel olarak üretir ve size `std::sort/std::set` için gereken toplam sıralamayı bedavaya verir — yeter ki üye sırası anlamsal sırayla örtüşsün. Gelecek ünitede `Vektor2B`'yi son iki yetenekle — akış operatörleri (`<<`/`>>`) ve indeksleme (`[]`) ile — donatarak onu "tam" bir değer tipine dönüştüreceksiniz.

3 • Katılım **Şimdi Siz Deneyin**

`Vektor2B` için `auto operator<=>(const Vektor2B&) const = default;` yazdınız. (a) `(2,3) < (4,1)` ifadesi ne verir? (b) Aynı `= default` yaklaşımını bir `Kesir(pay, payda)` için kullansanız, `1/2 < 1/3` ne verir ve bu doğru mudur?

Geribildirim. (a) `true`. Sözlüksel karşılaştırma önce 0. bileşene bakar; `2 < 4` olduğu için sonuç orada belirlenir, ikinci bileşene (`3` ile `1`) hiç bakılmaz. (b) `= default` önce `pay`'a bakar: `1 == 1`; sonra `payda`'ya bakar: `2 < 3`. Böylece sözlüksel sonuç `1/2 < 1/3` `true` çıkar — oysa sayıca `1/2 > 1/3`'tür, yani sonuç *yanlıştır*. `Kesir` için `<=>` elle, çapraz çarpma (`a*d` ile `c*b`) mantığıyla yazılmalıdır.

Kendinizi Sınayın 5.3

`<=>`'i `= default` ile tanımlamanın avantajı nedir, ve hangi durumda elle (custom) bir `<=>` yazmanız gerekir? Yanıtınızda `= default`'ın karşılaştırmayı hangi sıraya göre ürettiğini ve bu sıranın anlamsal sırayla örtüşmediği bir örnek (örneğin bir `Kesir`) verildiğinde neden yanlış sonuç doğacağını açıklayın.

5 • İzleme — Aktarım — Başka Nerede?

`= default`'ın "üye bildirim sırası = anlamsal sıra" varsayımı her tip için ayrıca sorgulanmalıdır. Kitaplık projesinde bir `Kitap`'ı başlığa ya da yıla göre sıralamak isterseniz, üye sırası (örneğin önce `isbn`, sonra `baslik`) istediğiniz sırayı vermeyebilir — o zaman `<=>`'i elle yazarsınız. Aynı biçimde, `double` alanlı bir tipte (ör. `NaN` içerebilen bir ölçüm) sonuç `strong_ordering` değil `partial_ordering` olur; hangi sonuç tipini alacağınızı üyelerinizin türü belirler. Kısacası `= default` güçlü bir kestirmedir, ama her tip için "bu sıra gerçekten doğru mu?" sorusunu sormadan güvenmeyin.

5.4 Akış Operatörleri ve İndeksleme: Değer Tipini Tamamlamak

1 • Ön-öğretim — Neden Önemli?

Bir `Vektor2B`'yi ekrana bastırmak için `std::cout << "(" << v.x() << ", " << v.y() << ")"` yazmak hem yorucu hem de hataya açıktır; ideal olan yalnızca `std::cout << v;` demektir. Benzer biçimde bir bileşene isimle değil, *konumla* (`v[0]`, `v[1]`) erişmek çoğu algoritmada gereklidir. Bu ünite, akış ve indeksleme operatörlerini ekleyerek `Vektor2B`'yi `int` kadar "doğal" davranan tam bir değer tipine dönüştürür.

1 • Ön-öğretim — Ön Bilgi

Bu ünite Ünite 1'in "sol işlenen sizin tipiniz değilse operatör serbest/`friend` olur" kuralını (burada `operator<<` ve `operator>>` ile somutlaşır), `private bilesenler_` alanını ve Hafta 4'ün `const`/const-olmayan üye ayrımını varsayar. Ünite 2–3'te kurduğunuz tüm operatörler de burada bir araya gelecektir. Ünite 1'de tanıtılan `friend` kavramı, bu ünitede ilk "gerçek" örneğini bulacak.

`Vektor2B` artık toplanabiliyor, çıkarılabiliyor, ölçeklenebiliyor ve karşılaştırılabiliyor. Ama bir eksiği hâlâ göze batıyor: onu ekrana yazdıramıyor, bileşenlerine numarasıyla erişemiyorsunuz. Bu ünitede son iki yeteneği — akış operatörleri (`<<`, `>>`) ve indeksleme (`[]`) — ekleyerek `Vektor2B`'yi, kullanıcının yerleşik bir tipten ayırt edemeyeceği kadar "doğal" davranan bir değer tipine dönüştüreceksiniz.

5.4.1 İndeksleme operatörü: konumsal erişim

indeksleme operatörü: `[]` ile bir nesnenin bileşenine konumsal erişim sağlayan operatör (*subscript operator*)

`v[0]` ve `v[1]` yazıp bir vektörün bileşenlerine numarasıyla erişmek istiyorsunuz. Bunu **indeksleme operatörü** sağlar. Sol işlenen (`v`) sizin tipiniz olduğu için `operator[]` bir üye fonksiyondur — ama burada beklenmedik bir incelik vardır: onu *iki kez*, biri `const` biri `const` olmayan sürümle yazarsınız:

```
int& operator[](int indeks);
int operator[](int indeks) const;
```

Neden iki sürüm? Çünkü `v[0]`'ı iki farklı bağlamda kullanmak istersiniz. Bazen *okumak* için (`int a = v[0];`), bazen de *yazmak* için (`v[0] = 10;`). Bu iki ihtiyaç, iki farklı dönüş tipi gerektirir; ikisini de ayrı ayrı okuyun.

Const olmayan sürüm (`int& operator[](int)`) bir *referans* döndürür. Referans döndürmesi kritiktir: `v[0] = 10;` yazdığınızda, `v[0]` çağrısı `bilesenler_[0]`'a bir referans verir, ve `= 10` bu referans üzerinden *doğrudan* diziye yazar. Yani `v[0]`, atamanın *sol tarafında* durabilen, üzerine yazılabilen bir ifadeye dönüşür — tıpkı yerleşik bir dizide `dizi[0] = 10;` yazdığınız gibi.

Const sürüm (`int operator[](int) const`) ise bir referans değil, bir *değer* (`int`) döndürür ve fonksiyonun kendisi `const` işaretlidir. Bu sürüm bir `const Vektor2B` üzerinde çağrılabilir — çünkü `const`'tur, nesneyi değiştirmez. Değer döndürdüğü için de `const` nesnenin bileşenini yalnızca *okumanıza* izin verir, üzerine yazmanıza değil; `const` bir vektöre `v[0] = 10;` yazmayı denerseniz derleyici buna izin vermez, ki bu tam da istenen davranıştır.

Gövdeleri kısacıktır; ikisi de diziye erişip ilgili sonucu döndürür:

```
49 int& Vektor2B::operator[](int indeks) {
50    // On kosul: indeks yalniz 0 ya da 1 olmalidir (hata yonetimi Hafta 13).
51    return bilesenler_[static_cast<std::size_t>(indeks)];
52 }
53
54 int Vektor2B::operator[](int indeks) const {
55    return bilesenler_[static_cast<std::size_t>(indeks)];
56 }
```

Gövdedeki yoruma dikkat: geçerli indeksler yalnızca 0 ve 1'dir; bu bir *ön koşuldur* ve yorumla belgelenir. Bu haftada, geçersiz bir indeks (örneğin `v[5]`) verildiğinde ne olacağını ele almazsınız — hata yönetimi ve istisnalar Hafta 13'ün konusudur. Hafta 2–4'ten tanıdığınız kurala burada da uyulur: ders notundaki hiçbir kesit gerçek bir tanımsız davranışı tetiklemez; yalnızca deseni ve ön koşulu açıklayıcı yorumla gösterir.

Analoji

İndekslemeyi bir apartmandaki daire numaralarına benzetebilirsiniz. Bir daireye ulaşmak için ona bir isim vermeniz gerekmez; sadece numarasını söylersiniz — "0 numaralı daire", "1 numaralı daire". Numara, kapıya doğrudan götüren konumsal bir adrestir. `v[0]`, `v[1]` tam olarak budur: bileşene ismiyle değil, konumuyla (numarasıyla) erişirsiniz. Const/const-olmayan ayrımını da bu benzetmeye ekleyin: const olmayan sürüm, dairenin *kapısını açıp içini değiştirmenize* izin veren bir anahtar gibidir (`v[0] = 10;`); const sürüm ise yalnızca kapının dışından numarayı *okumanıza* izin verir, içeri girip değiştirmenize değil.

5.4.2 Akış operatörü <<: nesneyi yazdırmak

akış operatörü: `std::cout`/`std::cin` gibi akışlarla bir tipi yazdırmayı/okumayı sağlayan <</>> operatörleri (*stream insertion/extraction operator*).

Şimdi `std::cout << v;` yazıp bir `Vektor2B`'yi `(2, 3)` biçiminde ekrana bastırmak istiyorsunuz. Bunu **akış operatörü** sağlar. İşte Ünite 1'de söz verilen kural burada somutlaşıyor. `std::cout << v` ifadesinin sol işleneni `std::cout`'tur; `std::cout` ise bir `std::ostream`'dir, sizin `Vektor2B`'niz *değildir*. Üye fonksiyon çağrısında sol işlenen örtük `this` olur — ama `this` sizin sınıfınızın nesnesi olmak zorundadır; `std::ostream` sizin sınıfınız olmadığına göre, `operator<<` üye *olamaz*. Serbest olmak zorundadır.

Dahası, bu serbest fonksiyon `Vektor2B`'nin `private` `bilesenler_` alanına doğrudan erişip yazdırmak ister. Ünite 1'den hatırlayın: sınıfın dışındaki bir fonksiyonun `private`'a erişmesi gerekiyorsa, sınıf ona `friend` izni verir. Bu yüzden akış operatörleri sınıf içinde `friend` olarak bildirilir:

```
36    friend std::ostream& operator<<(std::ostream& cikis, const Vektor2B& v);
37    friend std::istream& operator>>(std::istream& giris, Vektor2B& v);
```

Bu iki bildirim sınıfın *içinde* durur (izni sınıf verir), ama bildirdikleri fonksiyonlar sınıfın üyesi *değildir* — serbest fonksiyonlardır, yalnızca `private`'a erişim ayrıcalığı tanınmıştır. Bu, klasik ve standart C++ desenidir; bu haftanın "gerçek" `friend` örneği budur (Ünite 2'nin serbest `operator*`'ı `friend` değildi, çünkü `private`'a hiç dokunmuyordu; burada ise doğrudan `bilesenler_`'e erişiliyor). `operator<<`'in gövdesini okuyun:

```
60 std::ostream& operator<<(std::ostream& cikis, const Vektor2B& v) {
61    cikis << '(' << v.bilesenler_[0] << ", " << v.bilesenler_[1] << ')';
62    return cikis;
63 }
```

Gövdeyi parça parça izleyin. Fonksiyon `v.bilesenler_`'e doğrudan erişebiliyor — çünkü `friend`'dir; `private` olmasına rağmen kapı ona açıktır. Vektörü `(x, y)` biçiminde `cikis` akışına yazar. En önemli ayrıntı son satırdır: fonksiyon `cikis`'ı (`std::ostream&`, yani akışın kendisini) referansla *geri döndürür*. Bu neden gerekli? Zincirleme yazdırmayı mümkün kılmak için. `std::cout << v1 << v2;` ifadesini düşünün: bu, soldan sağa `(std::cout << v1) << v2` biçiminde işler; `std::cout << v1` önce çalışır ve `std::cout`'un kendisini geri döndürmelidir ki ardından gelen `<< v2` aynı akış üzerinde devam edebilsin. Akışı referansla döndürmek, işte bu zincirlemeyi sağlayan standart kalıptır.

5.4.3 Akış operatörü >>: nesneyi okumak

Aynı gerekçelerle `operator>>` de serbest ve `friend` olur: sol işlenen yine bir akıştır (`std::istream`), sizin tipiniz değildir; ve okunan değerleri `private` `bilesenler_`'e doğrudan yazması gerekir. `<<`'ten tek anlamlı farkı, sağ işlenenini `const` *olmayan* bir referansla (`Vektor2B& v`) almasıdır — çünkü `>>`, okuduğu değerleri `v`'ye *yazar*, dolayısıyla `v`'yi değiştirir:

```
67 std::istream& operator>>(std::istream& giris, Vektor2B& v) {
68    giris >> v.bilesenler_[0] >> v.bilesenler_[1];
69    return giris;
70 }
```

Gövde iki tam sayıyı `giris` akışından okuyup `v`'nin bileşenlerine yazar ve — tıpkı `<<` gibi — akışı geri döndürür.

Analoji

Akış operatörlerini bir borudaki suyun akış yönüne benzetebilirsiniz. `<<`, değeri akışın *içine doğru* akıtır — vektörü alıp `std::cout` borusundan ekrana gönderir. `>>` ise değeri akıştan *dışarı çeker* — girdi akışından okuyup vektöre doldurur. İki sembolün oku (`<<` sola, `>>` sağa) tam da bu akış yönünü gösterir: `cout <<` `v`'de ok `v`'den `cout`'a doğrudur (dışarı yazma), `cin >> v`'de ok `cin`'den `v`'ye doğrudur (içeri okuma). Koda eşleyin: `<<`'in gövdesi `v`'yi *okuyup* akışa yazar (bu yüzden `const Vektor2B&`), `>>`'in gövdesi ise akıştan okuyup `v`'ye *yazar* (bu yüzden `const` olmayan `Vektor2B&`).

Bu iki operatörün üye *olamayıp* serbest `friend` olma gerekçesi, aslında Ünite 1'in Şekil 5.1'inde sağ panelde çizilen durumun ta kendisidir: sol işlenen `std::ostream` olduğu için fonksiyon sınıfın dışında serbest durur, ve `private` alana erişmek için

kesikli okla gösterilen `friend` bağını kurar. İsterseniz o şekle bir daha bakıp bu üniteyle örtüştüğünü görün.

Şimdi `[]`, `<<` ve `>>`'i gerçek kullanımda izleyin. Önce indeksleme:

```
42  Vektor2B d(5, 7);
43  std::cout << "d[0]=" << d[0] << ", d[1]=" << d[1] << '\n';
44  d[0] = 10;
45  std::cout << "d[0]=10 sonrasi d=" << d << '\n';
```

Bu satırlar `d = (5, 7)` için `d[0]` ve `d[1]`'i okur, sonra `d[0] = 10;` ile ilk bileşeni yerinde değiştirir — bu atamanın çalışması, const olmayan `operator[]`'in referans döndürmesi sayesindedir. Ardından akış operatörleri:

```
49  Vektor2B e(1, 2);
50  std::cout << "operator<< ile e=" << e << '\n';
51
52  std::istringstream giris("7 9");
53  Vektor2B f;
54  giris >> f;
55  std::cout << "\"7 9\" metninden okunan f=" << f << '\n';
```

`std::cout << e;` vektörü `(1, 2)` biçiminde yazdırır. İkinci kısım öğreticidir: gerçek klavye girdisi beklemek yerine, sabit bir `"7 9"` metnini bir `std::istringstream`'e koyup ondan `>>` ile okursunuz. Bir `std::istringstream`, bir metni tıpkı bir girdi akışıymış gibi okumanızı sağlar; böylece demo hiç kullanıcı beklemeden, her çalıştırmada aynı deterministik sonucu (`f = (7, 9)`) üretir — `operator>>`'in klavyeden okuyanla birebir aynı fonksiyon olduğunu, yalnızca akış kaynağının değiştiğini görürsünüz.

5.4.4 "Tam" bir değer tipi

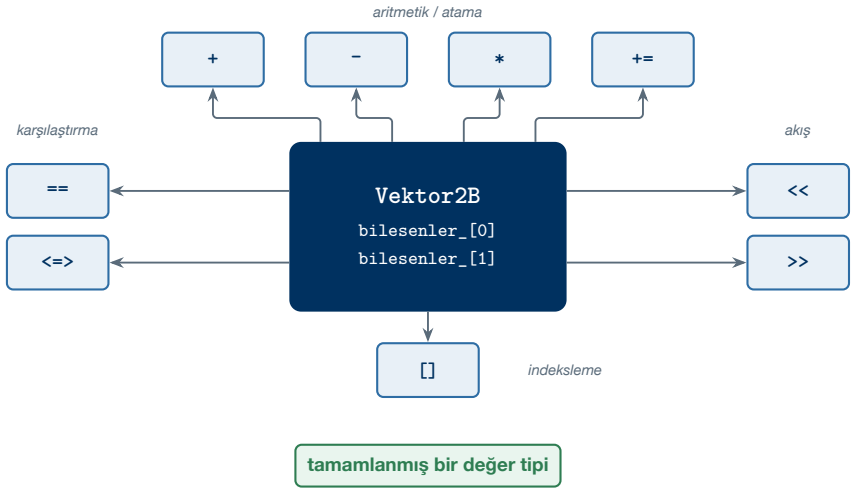
Bu haftanın yolculuğu burada tamamlanıyor. `Vektor2B`, Hafta 1–2'nin `topla(a, b)` çağrısıyla toplanan sade bir yapıdan başladı; adım adım her operatörle donatıldı. Şimdi geriye bakın: `Vektor2B` artık `+`, `-`, `*` (üye ve serbest), `+=`, `==`, `<=>` (`= default`), `<<`, `>>` ve `[]` operatörlerinin tamamına sahip. Bu, onu bir **değer tipi** yapar: kullanıcı açısından `Vektor2B`, bir `int` kadar doğal davranır — toplanır, karşılaştırılır, sıralanır, yazdırılır, indekslenir.

değer tipi: yerleşik tipler (`int` vb.) gibi doğal biçimde kopyalanan, karşılaştırılan, yazdırılan kullanıcı tanımlı tip (*value type*).

Analoji

Tamamlanmış bir değer tipini, yerleşik bir sayı gibi davranan kendi aracınıza benzetebilirsiniz. Bir `int` nasıl toplanabiliyor, karşılaştırılabiliyor, yazdırılabiliyorsa; operatörleriyle donatılmış `Vektor2B` de kullanıcı için aynı doğallıkta davranır. Kodu okuyan biri `v1 + v2`, `v1 == v2`, `std::cout << v1` satırlarını görür ve `v1`'in yerleşik bir tip mi yoksa sizin yazdığınız bir sınıf mı olduğunu ayırt edemez — "kendi `int`'inizi" tasarlamış olursunuz. İşte operatör yüklemenin nihai amacı budur.

Şekil 5.3 bu tamamlanmış tipi görsel olarak özetliyor: merkezde `Vektor2B` sınıfı (içinde `bilesenler_[0]`, `bilesenler_[1]` alanlarıyla) ve çevresinde bu hafta yüklediğiniz operatörler — aritmetik/atama (`+`, `-`, `*`, `+=`), karşılaştırma (`==`, `<=>`), akış (`<<`, `>>`) ve indeksleme (`[]`) — her biri merkez kutuya bir okla bağlı.



Şekil 5.3: Operatörleriyle donatılmış `Vektor2B`: aritmetik/atama, karşılaştırma, akış ve indeksleme operatörleri bir araya gelerek "tamamlanmış bir değer tipi" oluşturur.

İpucu

Bu haftanın desenini Hafta 3'ün `kitaplik::Kitap`'ına da uygulayabilirsiniz. `Kitap` şu an yalnızca bir `yazdir() const` üye fonksiyonuna sahip; öğrendiğiniz desenle ona serbest bir `friend operator<<` ekleyip `yazdir()`'in yerini alacak biçimde `std::cout << kitap;` yazımını mümkün kılabilir, başlığa ya da yıla göre sıralamak için de bir `operator<=>` tanımlayabilirsiniz. Ancak `Kitap` için `= default` muhtemelen yeterli olmaz — Ünite 3'ün notunu hatırlayın: üye sırasına göre karşılaştırma anlamsal sırayla örtüşmeyebilir (örneğin önce yıla göre sıralamak isterseniz, üyelerin bildirim sırasını buna uydurmanız ya da `<=>`'i elle yazmanız gerekebilir). Bu hafta `kitap.hpp`/`kitap.cpp` dosyalarına dokunmuyorsunuz; bu yalnızca ileriye dönük bir fikir olarak kalıyor.

Bu ünitede gördüğünüz fikir kısaca şudur: `operator[]`'i, const nesnelerde de okunabilsin diye biri `const` biri const-olmayan iki sürümle üye olarak yazarsınız; `operator<<`/`operator>>`'i ise sol işlenenleri akış tipi olduğu için serbest, `private`'a eriştikleri için `friend` olarak yazar ve zincirlemeyi sağlamak için akışı referansla döndürürsünüz. Böylece `Vektor2B`, yerleşik bir tip kadar doğal davranan tam bir değer tipine dönüştü. Gelecek hafta tek bir tipi tamamlamaktan öteye geçip birden çok *ilişkili* tipi — `Materyal` ve ondan türeyen `BasiliKitap`, `EKitap`, `SesliKitap`'ı — taban/türetilmiş sınıflar ve kompozisyonla nasıl organize edeceğinizi (Kalıtım ve Kompozisyon) göreceksiniz.

3 • Katılım ⌨ Şimdi Siz Deneyin

`Vektor2B` için indekslemeyi tek bir `int& operator[](int)` ile mi bırakırsınız, yoksa ikinci bir sürüm mü eklersiniz? Bir `const Vektor2B kv;` nesnesi için `std::cout << kv[0];` satırının derlenmesi için ne gerekir?

Geribildirim. Tek non-const sürüm yetmez: bir `const` nesnede yalnızca `const` üye fonksiyonlar çağrılabilir. Bu yüzden ikinci bir `int operator[](int) const` sürümü (salt-okunur, `int` döndürür) eklersiniz; `kv[0]` bu `const` sürümü çağı-

rır. Non-const sürüm ise `int&` döndürerek `v[0] = 10;` gibi değiştirmeye izin verir. Derleyici, nesnenin `const`'luğuna bakarak iki sürümden uygun olanı kendisi seçer.

Kendinizi Sınayın 5.4

`operator[]` için neden hem `const` hem de const olmayan iki ayrı aşırı yükleme tanımlarsınız? Yanıtınızda iki sürümün dönüş tiplerinin (`int&` ve `int`) neden farklı olduğunu ve her birinin hangi kullanımı (okuma / yazma, const nesne / const olmayan nesne) mümkün kıldığını açıklayın.

5 • İzleme — Aktarım — Başka Nerede?

Artık `Vektor2B`, `int` gibi davranan tam bir değer tipidir. Aynı `operator<</friend` deseniyle Hafta 3'ün `kitaplik::Kitap`'ına, `yazdir()`'in yerini alacak bir `operator<<` eklenebilir (bu hafta `kitap.hpp`/`kitap.cpp` değişmez; bu yalnızca ileriye dönük bir nottur). "Değer tipini operatörlerle tamamlama" becerisi kalıcıdır: gelecek hafta `Materyal` hiyerarşisini (`BasiliKitap`, `EKitap`, `SesliKitap`) kurarken, bu tiplerin de doğal biçimde yazdırılıp karşılaştırılabilmesi için burada öğrendiğiniz desenleri doğrudan yeniden kullanacaksınız.

Bölüm Özeti

Özet

Geçen hafta kaynaklarınızı Rule of 3 ve akıllı işaretçilerle güvenle kopyalamayı öğrendiniz; bu hafta o nesnelerin dilin yerleşik operatörleriyle doğal biçimde kullanılmasını, yani operatör yüklemeyi ele aldınız. Anlatının ekseni Hafta 1–2'nin `geometri::Nokta`'sıydı: serbest `topla(a, b)` çağrısından, aynı ad uzayına kız kardeş bir tip olarak eklenen `geometri::Vektor2B`'nin `a + b` yazımına geçiş. İlk ünitede operatör yüklemenin ne olduğunu — dilin sembollerini kendi tipiniz için özel isimli fonksiyonlarla anlamlandırmak — ve iki temel yolu gördünüz: sol işlenen sizin tipinizse operatör üye olabilir (örtük `this` sol işlenendir), değilse serbest olmak zorundadır; serbest bir fonksiyon `private`'a erişmesi gerekiyorsa sınıf ona `friend` izni verir. `Vektor2B` iskeletini kurdunuz ve ilk üye `operator+`'ı yazdınız. İkinci ünite aritmetiği tamamladı: `operator-` ve üye `operator*(int)` eklendi; `3 * v`'nin neden üye operatörle çalışmadığını (sol işlenen `int`'tir) ve bunun yalnızca `public` arayüzü kullandığı için `friend` olmayan bir serbest `operator*` ile nasıl çözüldüğünü gördünüz. `+=`'nin nesneyi yerinde güncelleyip `*this`'i referansla döndürdüğünü, salt okunur operatörlerin `const` olması gerektiğini ve yeni nesne üreten operatörlerin — sarkan referanstan kaçınmak için — değerle döndürülmesi gerektiğini öğrendiniz. Üçüncü ünite karşılaştırmayı ele aldı: C++20'nin `<=>` operatörü, `= default` ile yazıldığında altı karşılaştırma operatörünü üye sırasına göre sözlüksel olarak türetir ve `std::sort`/`std::set` için gereken toplam sıralamayı bedavaya verir — yeter ki üye sırası anlamsal sırayla örtüşsün (bir `Kesir`'de örtüşmez, elle yazmak gerekir). Dördüncü ünite değer tipini tamamladı: `operator[]`'i const/const-

olmayan çiftle üye olarak, akış operatörlerini (`<<`, `>>`) serbest `friend` olarak yazdınız ve akışı referansla döndürerek zincirlemeyi sağladınız. Bu hafta tek bir değer tipini (`Vektor2B`) operatörleriyle tamamladınız; gelecek hafta birden çok ilişkili tipi taban/türetilmiş sınıflar ve kompozisyonla (Kalıtım ve Kompozisyon) organize etmeyi göreceksiniz.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — `operator<<` neden üye olamaz). Bir üye operatör fonksiyonu çağrıldığında sol işlenen daima örtük `this`'tir; yani `a + b` ifadesi `a.operator+(b)` biçiminde çağrılır ve bu ancak soldaki nesne (`a`) sizin sınıfınızın bir örneğiyse mümkündür. `std::cout << v` ifadesinde ise sol işlenen `std::cout`'tur, ve `std::cout` bir `std::ostream`'dir — sizin `Vektor2B` sınıfınız değildir. Dolayısıyla `operator<<` üye olsaydı sol işleneni bir `std::ostream` kabul etmesi gerekirdi, ama `std::ostream` sizin yazdığınız bir tip olmadığı için ona üye fonksiyon ekleyemezsiniz. Bu koşul sağlanmadığından `operator<<` üye *olamaz*, ad uzayı kapsamında bir serbest fonksiyon olmak zorundadır; ayrıca sınıfın `private bilesenler_` alanına doğrudan erişip yazdırması gerektiği için, sınıf içinde `friend` olarak bildirilir — serbest kalıp `private`'a erişme ihtiyacı, tam da `friend`'in var oluş nedenidir.

Soru (Ünite 2 — `operator+` neden değer döndürür). `operator+`, iki işleneninden de bağımsız, *yeni* bir sonuç üretir; bu sonuç fonksiyonun içinde oluşturulan yerel bir `Vektor2B` değişkenidir ve fonksiyon bittiği anda kapsam dışına çıkıp yok olur. Eğer bu yerel değişkene bir referans (`Vektor2B&`) döndürseydiniz, çağıran taraf fonksiyon bittikten sonra artık *var olmayan* bir nesneye bakan bir referans elde ederdi — buna sarkan (dangling) referans denir ve okunması tanımsız davranıştır. Bu yüzden yeni bir nesne üreten `operator+` (ve aynı gerekçeyle `operator-`, `operator*`) sonucu *değerle* (by value) döndürür; sonuç çağırana kopyalanır ve güvenle yaşamaya devam eder. `operator+=` ise bu sorundan muaftır: o yeni bir nesne üretmez, zaten var olan `*this`'i günceller ve onun kendisini döndürür — `*this` fonksiyondan sonra da yaşadığı için ona referans döndürmek güvenlidir, hatta zincirlemeyi mümkün kıldığı için tercih edilir.

Soru (Ünite 3 — `= default` avantajı ve ne zaman elle yazılır). `<=>`'i `= default` ile tanımlamanın avantajı, tek bir satırdan altı karşılaştırma operatörünü (`<`, `<=`, `>`, `>=` ve `==`'le birlikte `==`, `!=`) bedavaya elde etmenizdir; derleyici karşılaştırmayı üyelerin bildirim sırasına göre sözlüksel olarak üretir ve size `std::sort`/`std::set` için gereken tutarlı toplam sıralamayı verir. Ama bunun bir koşulu vardır: üyelerin bildirim sırasındaki sözlüksel karşılaştırma, tipin *anlamsal/matematiksel* sırasıyla örtüşmelidir. `Vektor2B`'de örtüşür (bileşenleri sözlüksel karşılaştırmak makuldür). Ama bir `Kesir(pay, payda)` düşünün: üye sırasına göre önce `pay`, sonra `payda` karşılaştırılır ve `1/2` ile `1/3` için sözlüksel sonuç `1/2 < 1/3` çıkar — oysa sayıca `1/2 > 1/3`'tür. Böyle durumlarda `= default` yanlış sonuç verir; `<=>`'i elle, doğru matematiksel mantıkla (örneğin çapraz çarpma) yazmanız gerekir.

Soru (Ünite 4 — `operator[]` neden iki sürüm). `operator[]`'i iki sürümle tanımlarsınız çünkü `v[0]`'ı iki farklı bağlamda kullanmak istersiniz: bir nesnenin bileşenine hem *yazmak* (`v[0] = 10;`) hem de yalnızca *okumak* (`int a = v[0];`) için.

Const olmayan sürüm bir referans (`int&`) döndürür; bu, çağrının atamanın sol tarafında durup diziye doğrudan yazabilmesini sağlar — `v[0] = 10;` işte bu referans sayesinde çalışır. Const sürüm ise bir değer (`int`) döndürür ve fonksiyonun kendisi `const`'tur; bu sayede bir `const Vektor2B` üzerinde de çağrılabilir, ama yalnızca okumaya izin verir — const bir nesnenin bileşenini değiştirmeye çalışmak (`const` vektöre `v[0] = 10;`) derlenmez. Kısacası const olmayan sürüm "değiştirilebilir nesnede yaz/oku", const sürüm ise "değiştirilemez nesnede yalnızca oku" ihtiyacını karşılar; ikisi birlikte, `[]`'in yerleşik dizilerdeki kadar doğal davranmasını sağlar.